

Machine Learning 30412 - 2025 Project

Lavinia Benetollo 3265865, Natali Dzneldze 3258252, Giulia Marcantonio 3307112

May 16, 2025

1. Introduction & Exploratory Data Analysis

In this project, we address the problem of predicting the market value of professional football players based on many different features spanning personal details, technical attributes, and club-related information. The task involves a supervised regression problem, where the goal is to estimate the continuous target variable `value_eur`, representing a player's market value in euros.

We began with exploratory data analysis (EDA) to gain a high-level understanding of our training dataset coming from file `train.csv`. The dataset provided includes information on 15,391 professional football players (rows), with 76 features (columns).

1.1 Missing values

We examined missing values and found `nation_jersey_number` (96.1%), `nation_position` (96.1%), `club_loaned_from` (94.3%), `player_tags` (94.3%), `goalkeeping_speed` (88.8%) and `player_traits` (51.6%) are the features missing more than 15% of the values. For all the other features with lower missing rate, we chose to handle missing data by filling numeric columns with the median, which is far less sensitive to skewed distributions and extreme outliers than the mean. Because many football-related variables are right-skewed, median imputation offers a simple robust strategy. Regarding categorical columns, we chose to tag missing values with flag "MISSING" which allows the learning algorithm to exploit patterns of missingness that may correlate with market value.

1.2 Identifying numerical/categorical and irrelevant features

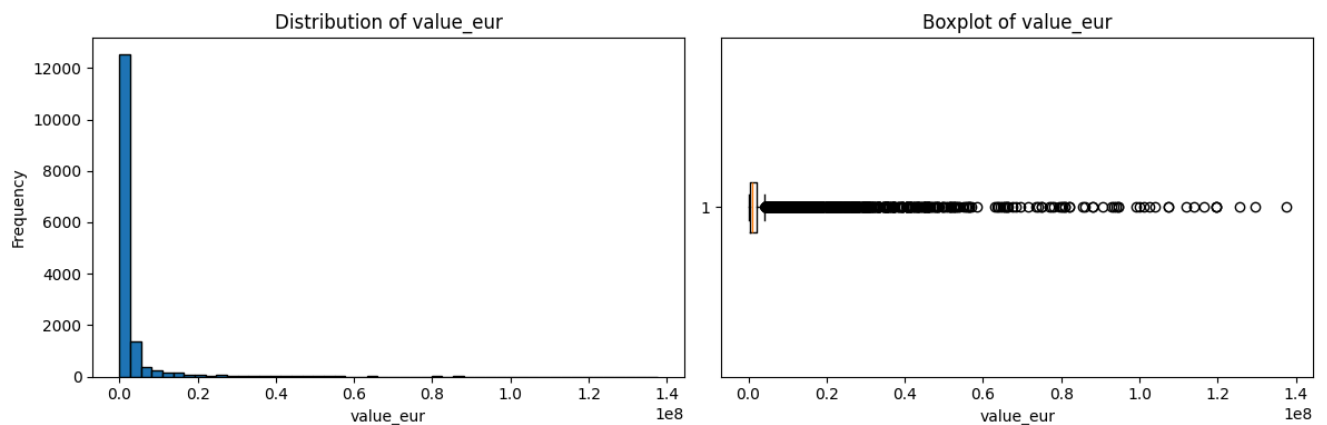
We identified 60 numerical and 16 categorical variables in the dataset. However, several of the numerical and categorical features in the dataset are considered *irrelevant* for modeling because they do not contain predictive information about a player's market value. These include:

- `id`, `club_team_id`: Unique identifiers that serve no predictive purpose.
- `short_name`, `long_name`: Names used for display; they are purely descriptive and redundant.
- `dob` (date of birth): Redundant after computing the `age` variable, which captures the same information more usefully.
- `nationality_id`: A coded form of `nationality_name`, which is already retained in a more interpretable form.
- `club_jersey_number`: Shirt number, which is arbitrary and unrelated to player skill or market value.
- `unnamed: 0`: A placeholder column resulting from file formatting, with no modeling utility.

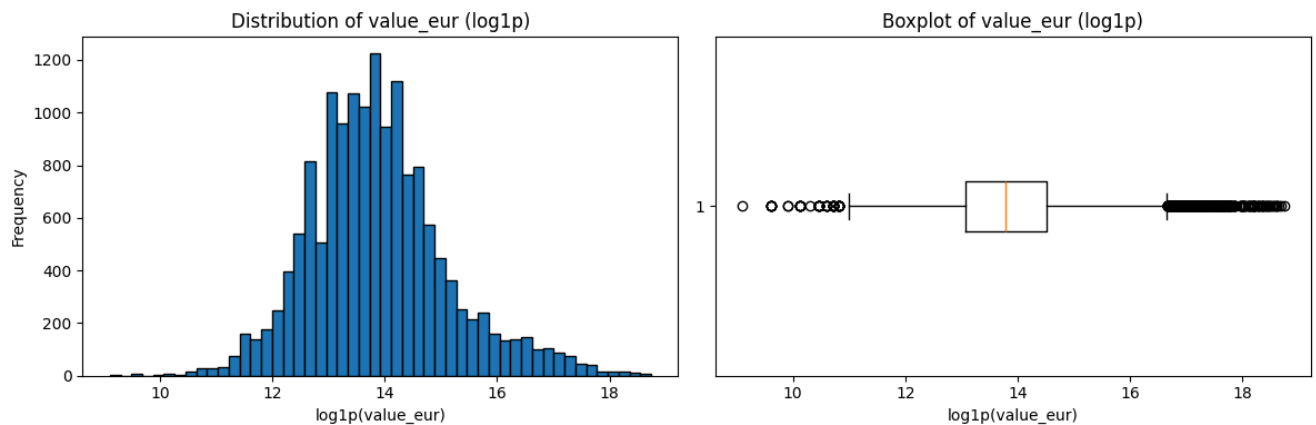
These features were flagged and subsequently removed to ensure only meaningful, predictive features are retained for modeling.

1.3 Transformation of the target feature

We analyzed the target variable *value_eur* and found it heavily right-skewed, with a few elite players worth far more than the rest. Such skewness can harm linear regression models by violating assumptions of normal errors and linearity. To fix this, we applied a log1p transformation, which compresses large values and reduces skewness.



After transformation, the distribution appeared more symmetrical and suitable for modeling. This helps stabilize variance, improve model performance, and make coefficients more reliable by focusing on relative value differences rather than extreme outliers.



1.4 Correlation analysis

To reduce multicollinearity and improve model stability, we performed correlation analysis among numeric variables, including *value_eur*. High correlation between predictors can cause redundancy and unstable coefficients in linear models. Using a 0.70 threshold, we flagged variable pairs with strong correlations. For each pair, we kept the variable more strongly linked to *value_eur* and dropped the weaker one, ensuring features are both informative and distinct. This process flagged 39 columns (*cols_to_drop*) for removal.

2. Preprocessing

We built a reusable *load_and_clean()* function to ensure consistent preprocessing across models. It loads the data, drops rows missing the target (*value_eur*), and applies a $\log(1 + x)$ transformation to *value_eur* to reduce skewness and stabilize variance. To handle missing data, we drop six features with the most missing values (*drop_missing*), remove *irrelevant_features* and for dates, we convert *club_joined* to year format and drop the original column. We kept just the year to avoid creating thousands of sparse dummy variables from full dates, which bloat models and risk overfitting, while still capturing the key temporal signal. We applied a *ColumnTransformer* to streamline preprocessing: it handles numerical and categorical features in parallel, ensuring that all data is properly scaled and encoded for modeling (cfr. 1.1).

3. Feature engineering

In our pipeline, we added a custom feature engineering step using the transformer *feat_eng()*. First, we drop *cols_to_drop* (cfr. 1.4) helping to reduce multicollinearity, simplify the model, and prevent overfitting.

We also dropped the *release_clause_eur* feature because it would introduce data leakage: since the release clause is often strongly correlated with a player's market value (*value_eur*), including it would give the model access to information that wouldn't be realistically available at prediction time, leading to overly optimistic results.

Additionally, we addressed the *player_positions* column, which originally contained highly specific role labels (e.g., "CB", "LW", "CAM"). Encoding these directly could have led to noisy and sparsely populated columns, especially due to overlapping or inconsistent position labels, increasing dimensionality and risk of overfitting. To solve this, we categorized player positions into four broader role groups: *is_defender*, *is_midfielder*, *is_forward*, and *is_goalkeeper*. This transformation not only simplified the data but also retained important information about player versatility, for instance, players listed with multiple roles would have multiple corresponding flags set to 1. After encoding, we dropped the original *player_positions* column to avoid redundancy.

We implement this using scikit-learn's *FunctionTransformer* (with *validate=False*) to integrate the function directly into our pipeline. Setting *validate=False* is crucial here because our transformer operates on pandas DataFrames rather than NumPy arrays, allowing us to use column names and perform label-based operations like dropping or assigning values. This design makes it easier to chain the transformer in a pipeline while keeping code readable and consistent with other preprocessing steps. Overall, this step helped improve both model accuracy and generalization.

4. Baseline Models

4.1 Linear regression

To establish our baseline, we trained a simple Linear Regression which provided a clear reference before testing more complex algorithms. We began by cleaning the data with our *load_and_clean()* function (cfr. 2), and then constructed a unified pipeline comprising three sequential stages: the preprocessing step (*preprocessor*), the feature engineering phase (*feat_eng*) and finally, the model fitting itself.

To ensure a robust assessment of performance, we employed five-fold cross-validation on the training set and reported both the mean and standard deviation of the RMSE across folds. The results were encouraging, with a training RMSE of € 2,079,666 ($\pm 148,163$) and a validation RMSE of € 2,198,263 ($\pm 608,412$). The validation error is only about 6% higher than the training error, indicating low overfitting and suggesting the model generalizes reasonably well. However, the high absolute error suggests that a simple linear model may not capture all complexities in the data. Exploring more advanced models or refining feature engineering would be logical next steps.

4.2 Lasso Implementation

As a second baseline, we ran **Lasso Regression on the already-reduced feature set** to check whether any further irrelevant predictors could be eliminated. Lasso's L_1 penalty automatically drives uninformative coefficients to zero, letting the model keep only the most predictive features.

We first tuned the regularization strength α via five-fold cross-validation, searching across 20 values on a logarithmic scale between $1e-5$ and $1e-3$ and found the best α to be 0.000089. Using this value, we trained the final Lasso model (again with cross-validation) and measured its error: on the training set, the model yielded an RMSE of € 2,143,019 ($\pm 136,332$), while on the held-out test set it achieved an RMSE of € 2,168,842 ($\pm 589,284$). Although Lasso's predictive accuracy is essentially **in line with** the unregularized Linear Regression (RMSEs differ by less than $\sim 2\%$), its coefficient-shrinkage confirms that no further irrelevant variables are present in the reduced set—so the model remains both interpretable and competitively accurate.

5. Complex models

In our exploration of more sophisticated approaches, we tested three tree-based algorithms, Decision Tree, Random Forest, and XGBoost. For all three implementations, we carried out hyperparameter tuning via grid search combined with five-fold cross-validation, evaluating all possible combinations of the parameters to minimize RMSE and using `n_jobs=-1` to leverage all available CPU cores and accelerate both model training and search.

5.1 Decision Tree

To train Decision Tree Regressor we created a pipeline similar to the previous ones, including feature engineering, preprocessing, and the model itself. Tuning `max_depth`, `min_samples_leaf`, and `ccp_alpha`, which control the tree's depth, the minimum number of samples per leaf, and pruning, respectively, the best hyperparameters found were `max_depth=20`, `min_samples_leaf=5` and `ccp_alpha=0.0`. With these settings, the best cross-validation RMSE was € 1,327,502 which is a significant improvement over the baseline models. This result suggests that the Decision Tree model with tuned parameters offers better predictive power than the simpler linear model, but further refinement and additional models may still improve performance.

5.2 Random Forest

To improve prediction accuracy, we implemented a Random Forest Regressor, an ensemble method that combines multiple decision trees to better capture non-linear patterns and reduce overfitting. We integrated it into a pipeline alongside our feature engineering and preprocessing steps. The tuned hyperparameters we found were `n_estimators=500`, a `max_depth=30`, and `min_samples_leaf=2`. The optimal configuration delivered a cross-validated RMSE of €961,981, our best result to date. However, training this

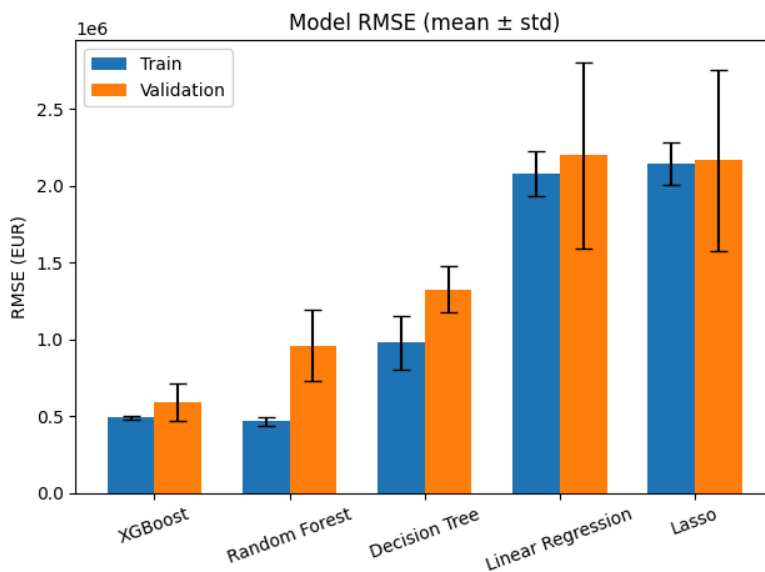
ensemble was computationally expensive, and despite its strong absolute performance, we later observed strong signs of overfitting. Given the lengthy tuning time required, we were unable to further mitigate this overfitting in our current experiment.

5.3 XGBoost

In the end, we implemented the XGBoost building a pipeline, similar to previous models, that integrates feature engineering, preprocessing, and the XGBRegressor. A full grid search with five-fold cross-validation identified the best hyperparameters as $n_estimators=500$, $max_depth=2$, $learning_rate=0.1$, $subsample=0.70$, and $colsample_bytree=1.00$. This configuration achieved a cross-validated RMSE of € 592,962, representing a substantial improvement over our earlier models.

Model Comparison and Selection

We evaluated each complex model under its best hyperparameter configuration using five-fold cross-validation, recording the training RMSE, the validation RMSE, and the percentage gap between them as a proxy for overfitting. As expected, our two linear baselines (Ordinary Least Squares and Lasso) show the higher RMSEs around €2.1 million and very small train–validation gaps, evidence of low overfitting but high bias. The Decision Tree reduces bias (training RMSE $\approx$ €1 million) but suffers from overfitting (validation RMSE $\approx$ €1.3 million), while Random Forest further lowers training error to roughly €450 000 at the cost of an even wider generalization gap (validation RMSE $\approx$ €950 000). XGBoost, however, strikes the best balance: it achieves the lowest errors (train $\approx$ €500 000, validation $\approx$ €600 000) and a moderate 20% gap, demonstrating that its sequential boosting framework can deliver both accuracy and controlled overfitting.



Conclusion

We selected XGBoost as our final model because it achieved the lowest validation RMSE, demonstrated only moderate overfitting, and showed consistent performance across folds. We then retrained it on the full training dataset and generated predictions for the test set, exporting the results to `submission.csv` for our final evaluation.